

Before we get into different options and configurations, let’s focus on some basic terms such as *terminal* and *shell*. In this section I’ll define the terminology and show you how to accomplish everyday tasks in the shell. We’ll also review modern commands and see them in action.

Terminals

We start with the terminal, or terminal emulator, or soft terminal, all of which refer to the same thing: a *terminal* is a program that provides a textual user interface. That is, a terminal supports reading characters from the keyboard and displaying them on the screen. Many years ago, these used to be integrated devices (keyboard and screen together), but nowadays terminals are simply apps.

In addition to the basic character-oriented input and output, terminals support so-called *escape sequences, or escape codes*, for cursor and screen handling and potentially support for colors. For example, pressing Ctrl+H causes a backspace, which deletes the character to the left of the cursor.

The environment variable TERM has the terminal emulator in use, and its configuration is available via infocmp as follows (note that the output has been shortened):

```
$ infocmp ❶
#      Reconstructed via infocmp from file: /lib/terminfo/s/screen-256color
screen-256color|GNU Screen with 256 colors,
    am, km, mir, msgr, xenl,
    colors#0x100, cols#80, it#8, lines#24, pairs#0x10000,
    acsc=++\,\,--..00``aaffgghhiijjkkllmmnnoppqqrrssttuuvvwxxyyz{[|]}~~,
    bel=^G, blink=\E[5m, bold=\E[1m, cbt=\E[Z, civis=\E[?25l,
    clear=\E[H\E[J, cnorm=\E[34h\E[?25h, cr=\r,
    ...
```

❶ The output of infocmp is not easy to digest. If you want to learn about the capabilities in detail, consult the **terminfo** database. For example, in my concrete output, the terminal supports 80 columns (cols#80) and 24 lines (lines#24) for output as well as 256 colors (colors#0x100, in hexadecimal notation).

Examples of terminals include not only xterm, rxvt, and the Gnome terminator but also new generation ones that utilize the GPU, such as **Alacritty**, **kitty**, and **warp**.

In “**Terminal Multiplexer**”, we will come back to the topic of the terminal.

Shells

Next up is the *shell*, a program that runs inside the terminal and acts as a command interpreter. The shell offers input and output handling via streams, supports variables, has some built-in commands you can use, deals with command execution and status, and usually supports both interactive usage as well as scripted usage (“**Scripting**”).

The shell is formally defined in **sh**, and we often come across the term **POSIX shell**, which will become more important in the context of scripts and portability.

Originally, we had the Bourne shell sh, named after the author, but nowadays it’s usually replaced with the **bash** shell—a wordplay on the original version, short for “Bourne Again Shell”—which is widely used as the default.

If you are curious about what you’re using, use the file -h /bin/sh command to find out, or if that fails, try echo \$0 or echo \$SHELL.

NOTE

In this section, we assume the bash shell (bash), unless we call it out explicitly.

There are many more implementations of sh as well as other variants, such as the Korn shell, ksh, and C shell, csh, which are not widely used today. We will, however, review modern bash replacements in “**Human-Friendly Shells**”.

Let’s start our shell basics with two fundamental features: streams and variables.

Streams

Let’s start with the topic of input (streams) and output (streams), or I/O for short. How can you feed a program some input? How do you control where the output of a program lands, say, on the terminal or in a file?

First off, the shell equips every process with three default file descriptors (FDs) for input and output:

- stdin (FD 0)
- stdout (FD 1)
- stderr (FD 2)

These FDs are, as depicted in **Figure 3-2**, by default connected to your screen and keyboard, respectively. In other words, unless you specify something else, a command you enter in the shell will take its input (stdin) from your keyboard, and it will deliver its output (stdout) to your screen.

The following shell interaction demonstrates this default behavior:

```
$ cat
This is some input I type on the keyboard and read on the screen^C
```

In the preceding example using `cat`, you see the defaults in action. Note that I used `Ctrl+C` (shown as `^C`) to terminate the command.

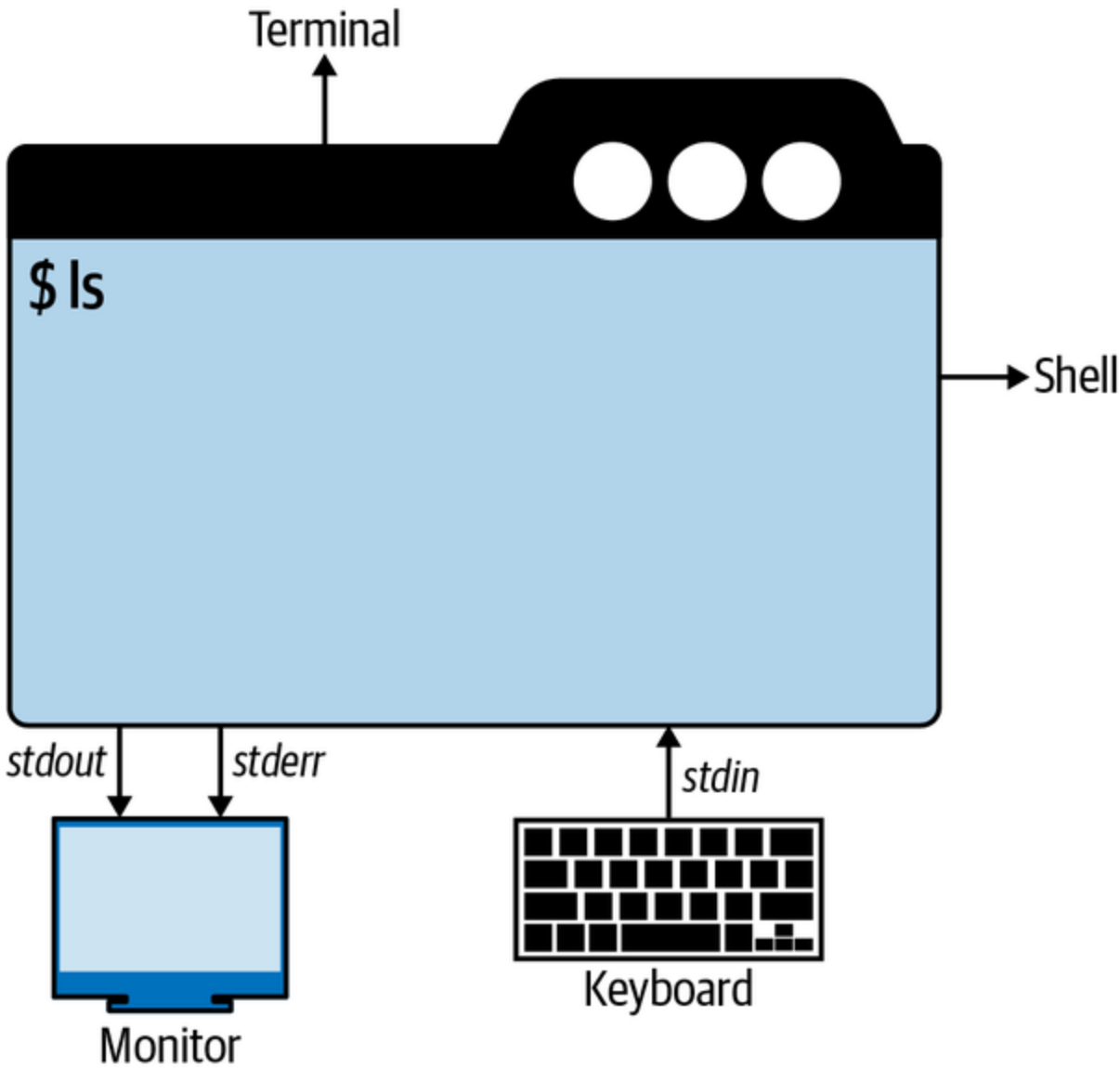


Figure 3-2. Shell I/O default streams

If you don’t want to use the defaults the shell gives you—for example, you don’t want `stderr` to be outputted on the screen but want to save it in a file—you can **redirect** the streams.

You redirect the output stream of a process using `$FD>` and `<$FD`, with `$FD` being the file descriptor—for example, `2>` means redirect the `stderr` stream. Note that `1>` and `>` are the same since `stdout` is the default. If you want to redirect both `stdout` and `stderr`, use `&>`, and when you want to get rid of a stream, you can use `/dev/null`.

Let’s see how that works in the context of a concrete example, downloading some HTML content via `curl`:

```
$ curl https://example.com &> /dev/null ❶

$ curl https://example.com > /tmp/content.txt 2> /tmp/curl-status ❷
$ head -3 /tmp/content.txt
<!doctype html>
<html>
<head>
$ cat /tmp/curl-status
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload   Total   Spent    Left   Speed
100 1256 100 1256    0     0  3187      0 --:--:-- --:--:-- --:--:-- 3195

$ cat > /tmp/interactive-input.txt ❸

$ tr < /tmp/curl-status [A-Z] [a-z] ❹
% total    % received % xferd  average speed  time    time     time  current
 dload  upload   total   spent    left   speed
100 1256 100 1256    0     0  3187      0 --:--:-- --:--:-- --:--:-- 3195
```

- ❶ Discard all output by redirecting both `stdout` and `stderr` to `/dev/null`.
- ❷ Redirect the output and status to different files.
- ❸ Interactively enter input and save to file; use `Ctrl+D` to stop capturing and store the content.
- ❹ Lowercase all words, using the `tr` command that reads from `stdin`.

Shells usually understand a number of special characters, such as:

Ampersand (&)

Placed at the end of a command, executes the command in the background (see also “Job control”)

Backslash (\)

Used to continue a command on the next line, for better readability of long commands

Pipe (|)

Connects stdout of one process with the stdin of the next process, allowing you to pass data without having to store it in files as a temporary place

PIPES AND THE UNIX PHILOSOPHY

While pipes might seem not too exciting at first glance, there’s much more to them. I once had a nice interaction with Doug McIlroy, the inventor of pipes. I wrote an article, “Revisiting the Unix Philosophy in 2018”, in which I drew parallels between UNIX and microservices. Someone commented on the article, and that comment led to Doug sending me an email (very unexpectedly, and I had to verify to believe it) to clarify things.

Again, let’s see some of the theoretical content in action. Let’s try to figure out how many lines an HTML file contains by downloading it using curl and then piping the content to the wc tool:

```
$ curl https://example.com 2> /dev/null | \ ❶
wc -l ❷
46
```

- ❶ Use curl to download the content from the URL, and discard the status that it outputs on stderr. (Note: in practice, you’d use the -s option of curl, but we want to learn how to apply our hard-gained knowledge, right?)
- ❷ The stdout of curl is fed to stdin of wc, which counts the number of lines with the -l option.

Now that you have a basic understanding of commands, streams, and redirection, let’s move on to another core shell feature, the handling of variables.

Variables

A term you will come across often in the context of shells is *variables*. Whenever you don’t want to or cannot hard-code a value, you can use a variable to store and change a value. Use cases include the following:

- When you want to handle configuration items that Linux exposes—for example, the place where the shell looks for executables captured in the \$PATH variable. This is kind of an interface where a variable might be read/write.
- When you want to interactively query the user for a value, say, in the context of a script.
- When you want to shorten input by defining a long value once—for example, the URL of an HTTP API. This use case roughly corresponds to a const value in a program language since you don’t change the value after you have declared the variable.

We distinguish between two kinds of variables:

Environment variables

Shell-wide settings; list them with env.

Shell variables

Valid in the context of the current execution; list with set in bash. Shell variables are not inherited by subprocesses.

You can, in bash, use export to create an environment variable. When you want to access the value of a variable, put a \$ in front of it, and when you want to get rid of it, use unset.

OK, that was a lot of information. Let’s see how that looks in practice (in bash):

```
$ set MY_VAR=42 ❶
$ set | grep MY_VAR ❷
_=MY_VAR=42

$ export MY_GLOBAL_VAR="fun with vars" ❸

$ set | grep 'MY_*' ❹
MY_GLOBAL_VAR='fun with vars'
_=MY_VAR=42

$ env | grep 'MY_*' ❺
MY_GLOBAL_VAR=fun with vars

$ bash ❻
$ echo $MY_GLOBAL_VAR ❼
fun with vars

$ set | grep 'MY_*' ❽
```

```
MY_GLOBAL_VAR='fun with vars'

$ exit 9
$ unset $MY_VAR
$ set | grep 'MY_*'
MY_GLOBAL_VAR='fun with vars'
```

- ❶ Create a shell variable called MY_VAR, and assign a value of 42.
- ❷ List shell variables and filter out MY_VAR. Note the _=, indicating it’s not exported.
- ❸ Create a new environment variable called MY_GLOBAL_VAR.
- ❹ List shell variables and filter out all that start with MY_. We see, as expected, both of the variables we created in the previous steps.
- ❺ List environment variables. We see MY_GLOBAL_VAR, as we would hope.
- ❻ Create a new shell session—that is, a child process of the current shell session that doesn’t inherit MY_VAR.
- ❼ Access the environment variable MY_GLOBAL_VAR.
- ❽ List the shell variables, which gives us only MY_GLOBAL_VAR since we’re in a child process.
- ❾ Exit the child process, remove the MY_VAR shell variable, and list our shell variables. As expected, MY_VAR is gone.

In **Table 3-1** I put together common shell and environment variables. You will find those variables almost everywhere, and they are important to understand and to use. For any of the variables, you can have a look at the respective value using `echo $XXX`, with XXX being the variable name.